

가천대학교 스마트시티학과 – 가상공간과 메타버스

Lecture 15 – City Economy and Policy Lab

Chris Ohk

utilForever@gmail.com

학습 목표

- Lecture 14의 재정 계산과 정책 비교를 기존 프로그램에 적용한다.
- 함수 6개를 작성해 세입 / 세출 / 부채를 계산하고 정책별 결과를 비교한다.
  - 코드 작성 → 단계별 검사 → 화면 확인 → 조건을 바꿔 실험

실행 준비

- 압축을 푼 스켈레톤 폴더에서 프로그램을 실행한다.
  - Windows: `py -m tkinter` → `py app.py`
  - macOS / Linux: `python3 -m tkinter` → `python3 app.py`
- `economy_rules.py`를 저장한 뒤 앱을 닫고 다시 실행한다.

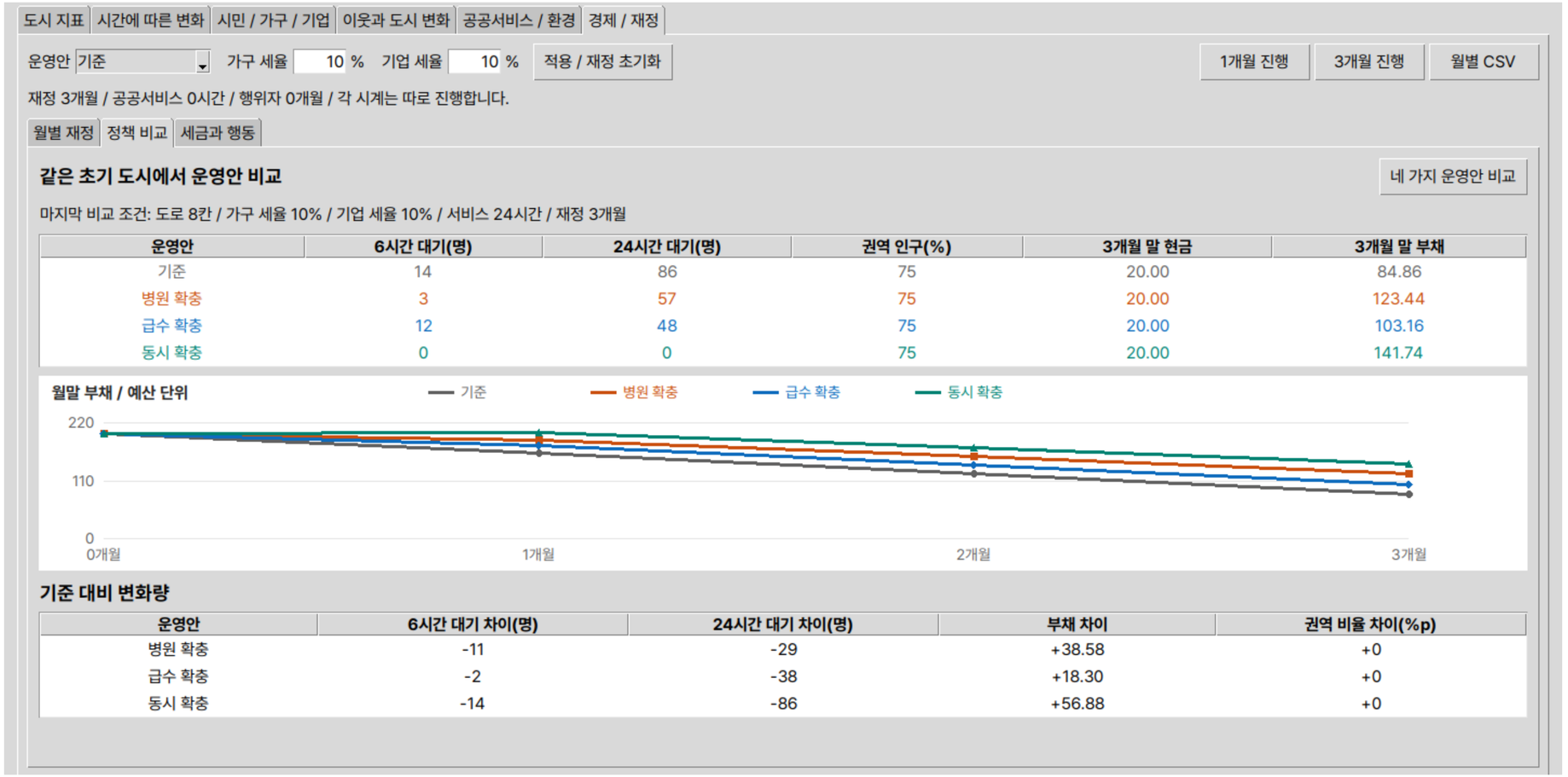
역할 나누기

- 제공된 코드에서 `economy_rules.py`의 TODO 1~6을 채운다.

파일	담당 내용	작성 여부
economy_rules.py	세금 / 비용 / 결산 / 정책 비교	직접 작성
economy_gui.py	재정 화면 / 그래프 / 버튼	제공
economy_support.py	월 진행 / 상태 저장 / 비교 실행	제공
checks_economy.py	단계별 검사 / 통합 검사	제공
app.py / 기존 도시 코드	실행 / 지도 / 행위자 / 공공서비스	제공

실습 완성 화면

- 같은 도시에서 네 정책의 진료 대기와 부채를 비교한다.



도시 재정의 자료구조

- `finance`에 현재 상태를 두고, 갱신 결과를 `history`에 기록한다.

자료	주요 항목	의미
`city / services`	지도 / 도로 / 대기 / 물	기존 도시 상태
`Finance`	`month / cash / debt`	현재 월 / 현금 / 부채
`finance`	`policy / capital_paid`	선택 정책 / 건설비 반영 여부
`finance["history"]`	월별 결과의 딕셔너리 목록	결산 과정과 잔액 기록

같은 도시에서 출발하기

- 기존 서비스 모형에 재정 계산에 필요한 가정만 더한다.

구분	초기 조건
서비스 모형	도로 8칸 / 인구 400명 / 대기 0명 / 물 10m³
경제 가정	취업자 200명 / 월 임금 5 / 기업 이익 200
재정 상태	현금 20 / 부채 200 / 확보할 현금 20
시간과 단위	서비스 : 시간 / 재정 : 월 / 금액 : 예산 단위

실습 1 : 세금 수입

- `tax_revenue(...)` 함수를 작성한다.
  - 취업자 수와 임금으로 가구 소득 합계를 구한다.
  - 가구 소득과 기업 이익에 각각 세율을 적용한다.
  - 소득 / 가구 세금 / 기업 세금 / 세금 합계를 반환한다.

소득에서 세금 계산하기

- 가구 세금 100과 기업 세금 20을 더하면 세금 수입은 120이다.

계산 항목	계산	반환값
가구 소득 합계	200×5	`income = 1,000`
가구 세금	$1{,}000 \times 0.10$	`household_tax = 100`
기업 세금	200×0.10	`firm_tax = 20`
세금 수입 합계	$100 + 20$	`revenue = 120`

세율과 이익의 경계값

- 이 모형에서는 적자 기업의 세금을 0으로 처리한다.

입력의 변화	기업 세금	전체 세금 수입
기업 이익 200 / 세율 10%	20	120
기업 이익 0 / 세율 10%	0	100
기업 이익 −50 / 세율 10%	0	100
가구, 기업 세율 모두 0%	0	0

직접 해보기 : 세금 수입

- `economy_rules.py`의 TODO 1을 완성해 보자.
  - `python checks_economy.py --stage 1`
  - 기본 조건의 결과 : 소득 1,000 / 세금 수입 120
- 기업 이익이 음수이면 기업 세금은 얼마가 되어야 할까?

세금 수입 해설

- 가구 소득과 기업 이익을 서로 다른 기준 금액으로 사용한다.

```python
def tax_revenue(employed, wage, firm_profit,
                income_tax_rate, profit_tax_rate):
    income = employed * wage
    household_tax = income * income_tax_rate
    firm_tax = max(0, firm_profit) * profit_tax_rate

    return {"income": income,
            "household_tax": household_tax,
            "firm_tax": firm_tax,
            "revenue": household_tax + firm_tax}
```

실습 2 : 매달 드는 비용

- `monthly_costs(...)` 함수를 작성한다.
  - 도로 수와 단가, 시설 수선비로 유지관리비를 구한다.
  - 서비스별 운영 예산을 모두 더한다.
  - 유지관리비 / 서비스 운영비 / 반복 비용을 반환한다.

도로와 시설의 반복 비용

- 도로 8칸과 서비스 운영 조건을 바탕으로 월 비용을 계산한다.

항목	계산	월 비용
도로 유지	8칸 × 2	16
기존 시설 수선	고정 가정	14
유지관리비 합계	16 + 14	30
서비스 운영	병원 24 + 급수 6 + 기타 20	50
반복 비용 합계	30 + 50	80

운영 규모가 바뀐 경우

- 확충한 시설의 운영비는 다음 달에도 계속 발생한다.

월 비용	기준	병원 확충	급수 확충	동시 확충
유지관리비	30	30	30	30
서비스 운영비	50	56	52	58
반복 비용	80	86	82	88

직접 해보기 : 반복 비용

- `economy_rules.py`의 TODO 2를 완성해 보자.
  - `python checks_economy.py --stage 2`
  - 기본 조건 : 유지관리비 30 / 서비스 운영비 50
- 도로가 9칸이면 유지관리비는 얼마가 될까?

반복 비용 해설

- 유지관리비와 서비스 운영비를 계산한 뒤 반복 비용으로 합친다.

```python
def monthly_costs(road_cells, road_unit_cost,
                  facility_maintenance, service_budgets):
    maintenance = (road_cells * road_unit_cost
                   + facility_maintenance)
    services = sum(service_budgets.values())

    return {"maintenance": maintenance,
            "services": services,
            "recurring": maintenance + services}
```

실습 3 : 건설비

- `construction_cost(...)` 함수를 작성한다.
  - 건설 항목별 수량과 단가를 곱한다.
  - 항목별 금액을 더해 총건설비를 반환한다.
  - 건설비를 반영할 시점은 제공 코드가 관리한다.

수량과 단가로 계산하기

- 동시 확충의 건설비는 병원 20과 급수 시설 12를 더한 32다.

항목	단가	동시 확충 수량	금액
road / 도로	20	0	0
hospital / 병원	20	1	20
water / 급수	12	1	12
합계			32

건설비는 한 번만 반영

- 사업이 완료되면 다음 달에는 건설 수량을 0으로 전달한다.

동시 확충	건설비 반영 여부	병원 / 급수 수량	다음 계산의 건설비
첫 달 시작	False	1 / 1	32
첫 달 결산 후	True	0 / 0	0
둘째 달 결산	True	0 / 0	0

직접 해보기 : 건설비

- `economy_rules.py`의 TODO 3을 완성해 보자.
  - `python checks_economy.py --stage 3`
  - 병원 확충 20 / 급수 확충 12 / 동시 확충 32
- 수량이 모두 0이면 어떤 값을 반환해야 할까?

건설비 해설

- 항목별 수량에 같은 항목의 단가를 곱해 누적한다.

```python
def construction_cost(quantities, unit_costs):
    total = 0

    for item, quantity in quantities.items():
        total += quantity * unit_costs[item]

    return total
```

실습 4 : 한 달의 수지

- `monthly_balance(...)` 함수를 작성한다.
  - 월초 부채에 월 이자율을 곱해 이자를 계산한다.
  - 세금 수입에서 유지비 / 운영비 / 이자 / 건설비를 뺀다.
  - 이번 달 이자와 월 수지를 반환한다.

흑자와 적자 계산하기

- 첫 달 기준 조건은 38의 흑자, 동시 확충은 2의 적자다.

항목 / 예산 단위	기준	동시 확충
세금 수입	120	120
유지관리비 / 운영비	30 / 50	30 / 58
이자 / 건설비	2 / 0	2 / 32
월 수지	$120 - 30 - 50 - 2 = 38$	$120 - 30 - 58 - 2 - 32 = -2$

이자는 월초 부채로 계산

- 이번 달 말의 차입과 상환은 다음 달 이자부터 반영된다.

기준 조건	월초 부채	월 이자율	이자
첫째 달	200	0.01	2
둘째 달	162	0.01	1.62
셋째 달	123.62	0.01	1.2362

직접 해보기 : 월 수지

- `economy_rules.py`의 TODO 4를 완성해 보자.
  - `python checks_economy.py --stage 4`
  - 첫 달 이자 2 / 기준 수지 38 / 동시 확충 수지 -2
- 부채가 00이면 이번 달 이자는 얼마일까?

월 수지 해설

- 차입과 원금 상환을 제외한 수입 / 비용 차이를 계산한다.

```python
def monthly_balance(revenue, maintenance, services,
                    debt, interest_rate, construction):
    interest = debt * interest_rate
    balance = (revenue - maintenance - services
               - interest - construction)

    return {"interest": interest, "balance": balance}
```

실습 5 : 현금과 부채

- `settle_finances(...)` 함수를 작성한다.
  - 정해 둔 현금을 확보한 뒤 남는 돈으로 부채를 갚는다.
  - 현금이 부족하면 필요한 금액만 빌린다.
  - 차입 / 상환 / 월말 현금 / 월말 부채를 반환한다.

흑자를 원금 상환에 사용

- 현금 20을 남기고 흑자 38로 원금을 갚는다.

계산 단계	계산	결과
상환에 쓸 수 있는 돈	$20 + 38 - 20$	38
원금 상환	$\min(200, 38)$	38
월말 현금	$20 + 38 - 38$	20
월말 부채	$200 - 38$	162

부족한 현금은 차입으로 보충

- 동시 확충의 첫 달에는 부족한 금액 2를 빌린다.

계산 단계	계산	결과
현금의 여유 또는 부족	$20 - 2 - 20$	-2
신규 차입	$\max(0, 2)$	2
월말 현금	$20 - 2 + 2$	20
월말 부채	$200 + 2$	202

부채를 모두 갚은 뒤

- 남은 부채까지만 상환하고 나머지는 현금으로 보유한다.

월초 현금 / 부채	월 수지	차입 / 상환	월말 현금 / 부채
20 / 5	+10	0 / 5	25 / 0
25 / 0	−2	0 / 0	23 / 0

직접 해보기 : 자금 결산

- `economy_rules.py`의 TODO 5를 완성해 보자.
  - `python checks_economy.py --stage 5`
  - 기준 조건 : 상환 38 / 월말 현금 20 / 월말 부채 162
- 남은 부채가 흑자보다 적으면 얼마까지 갚아야 할까?

자금 결산 해설

- 현금과 부채에 차입 / 상환을 각각 반영한다.

```python
def settle_finances(cash, debt, balance, reserve):
    available = cash + balance - reserve
    borrowing = max(0, -available)
    repayment = min(debt, max(0, available))

    return {"borrowing": borrowing,
            "repayment": repayment,
            "cash": cash + balance + borrowing - repayment,
            "debt": debt + borrowing - repayment}
```

실습 6 : 기준과의 차이

- `compare_policy(...)` 함수를 작성한다.
  - 같은 시점의 정책 결과에서 기준 결과를 뺀다.
  - 6시간 / 24시간 후의 대기 인원과 권역 비율을 비교한다.
  - 3개월 말의 부채와 현금을 비교한다.
  - 서로 다른 단위의 차이를 한 점수로 더하지 않는다.

비교 실험의 공통 조건

- 정책마다 도시 상태를 새로 복사하고 같은 기간을 계산한다.

비교 항목	모든 대안의 공통 조건
같은 초기 서비스	대기 0명 / 물 10m³ / 같은 지도와 수요
같은 초기 재정	현금 20 / 부채 200 / 같은 세율과 경제활동
같은 평가 시점	서비스 6시간 / 24시간 / 재정 3개월
정책으로 바꿀 값	병원 / 급수 운영 규모 / 건설비 / 운영비

급수 확충과 기준의 차이

- 대기는 줄지만 3개월 말 부채는 기준보다 많다.

항목	기준	급수 확충	정책 − 기준
6시간 대기 / 명	14	12	−2
24시간 대기 / 명	86	48	−38
3개월 말 부채	84.8562	103.1576	+18.3014
권역 인구 비율	0.75	0.75	0 / 0%p

직접 해보기 : 정책 비교

- `economy_rules.py`의 TODO 6을 완성해 보자.
  - `python checks_economy.py --stage 6`
  - 급수 확충 : 24시간 대기 −38 / 부채 +18.3014
- 기준과 기준을 비교하면 각 항목의 차이는 얼마일까?

정책 비교 해설

- 같은 항목끼리 빼고 차이를 새 딕셔너리에 담는다.

```python
def compare_policy(baseline, alternative):
    differences = {}

    for key in ("queue6", "queue24", "debt", "cash", "coverage"):
        differences[key] = alternative[key] - baseline[key]

    return differences
```

한 달의 갱신 순서

- 계산이 모두 끝난 뒤 현금과 부채를 다음 상태에 반영한다.

순서	읽는 값 / 계산 결과
1. 세금과 비용	경제활동 / 도로 수 / 서비스 예산 / 건설 수량
2. 월 수지	세금 − 유지비 − 운영비 − 이자 − 건설비
3. 자금 결산	월초 현금 / 부채 / 차입 / 상환 / 월말 잔액
4. 기록과 상태 갱신	`month` / `cash` / `debt` / `capital_paid` / `history`

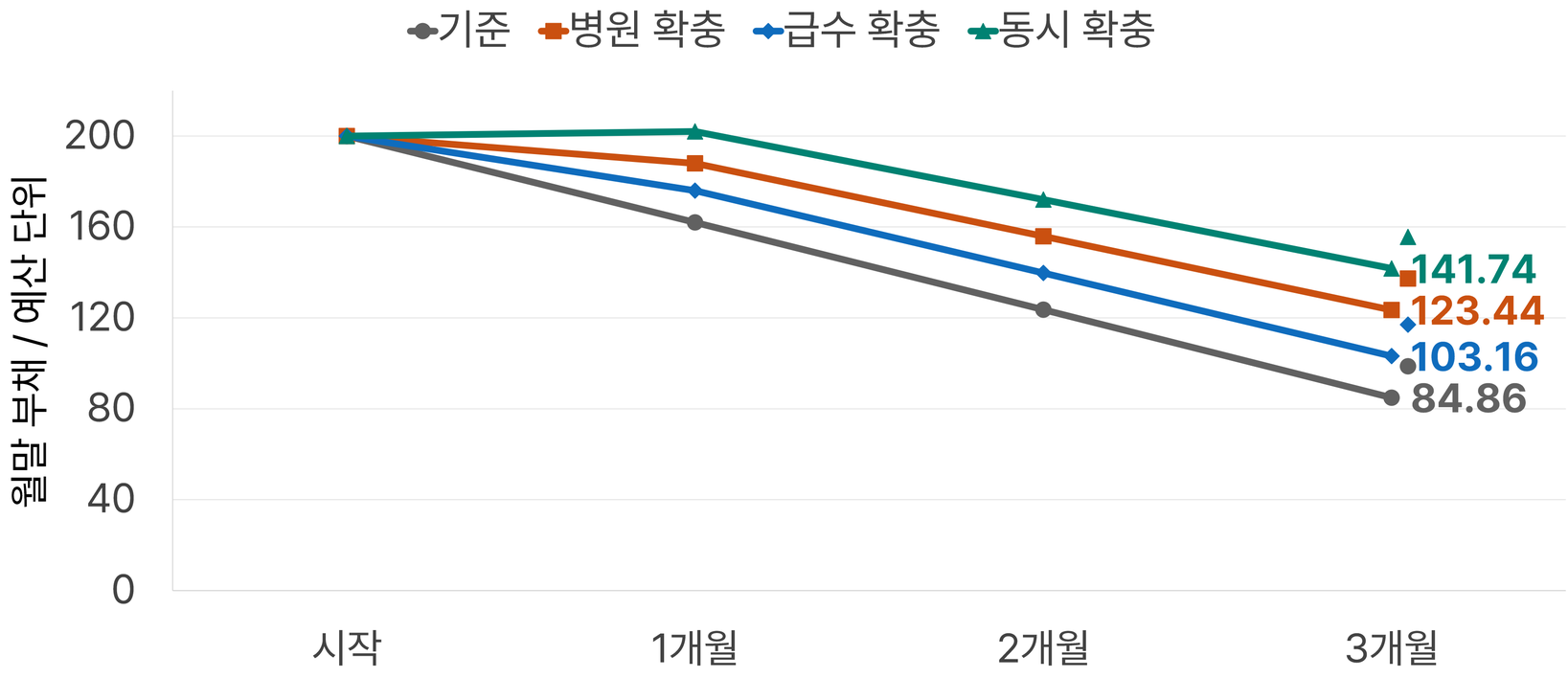
네 정책 비교 실행

- 기본 세율을 유지하고 네 정책을 같은 조건에서 실행한다.

설정	기준	병원 확충	급수 확충	동시 확충
병원 기본 능력 / 명/시간	8	10	8	10
물 유입 / m^3/시간	6	6	8	8
월 서비스 운영비	50	56	52	58
첫 달 건설비	0	20	12	32

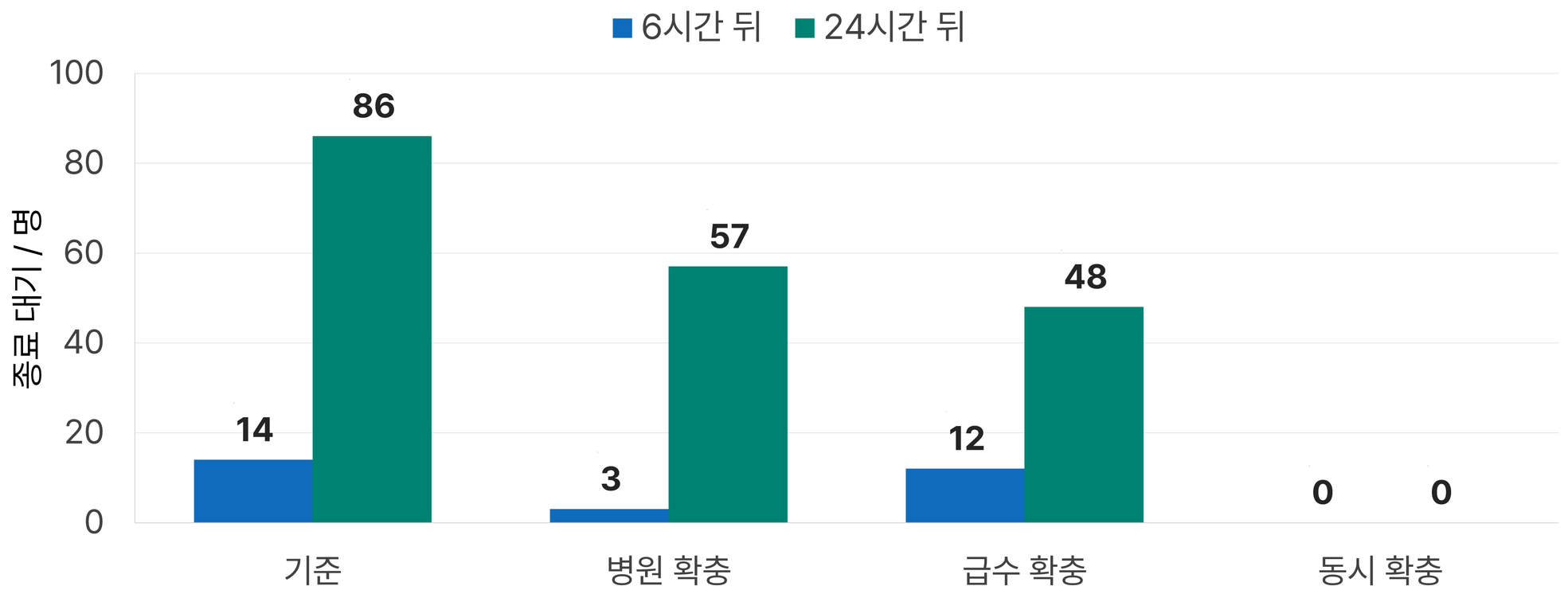
3개월 동안의 부채

- 현금이 모두 20이어도 남은 부채는 정책마다 다르다.



평가 시점에 따른 차이

- 6시간 뒤와 24시간 뒤에는 확충안의 순위가 달라진다.



서비스 성과와 재정 부담

- 대기와 부채, 권역을 함께 보고 정책을 선택한다.

결과	기준	병원 확충	급수 확충	동시 확충
24시간 대기 / 명	86	57	48	0
3개월 말 부채	84.86	123.44	103.16	141.74
권역 인구 비율	75%	75%	75%	75%
24시간 미배정 / 건	48	48	48	48

가구 세율을 바꾸면

- 소득과 이익을 고정하고 가구 세율만 10%에서 9%로 바꾼다.

비교 항목	가구 세율 10%	가구 세율 9%
가구 세금	100	90
세후 가구 소득	900	910
전체 세금 수입	120	110
기준 운영의 3개월 말 부채	84.8562	115.1572

세금과 기업 이전 판단

- 제공된 F1 사례에서 세후 이익에 따른 이전 유인을 확인한다.

이익세율	6개월 이전 순편익 / 만원	해석
10%	$6 \times (260 - 150) \times 0.9 - 500 = 94$	이전할 유인이 있음
30%	$6 \times (260 - 150) \times 0.7 - 500 = -38$	현재 위치 유지에 유리

기록과 전체 상태 저장

- 결과를 분석할 CSV와 다시 실행할 JSON을 구분한다.

저장 자료	주요 내용	용도
월별 CSV	세금 / 비용 / 이자 / 차입 / 상환 / 잔액	결과 분석
도시 JSON	지도 / 행위자 / 서비스 / 재정의 전체 상태	복원과 이어 실행
정책 비교 결과	정책별 기간 / 대기 / 부채 / 기준 대비 차이	대안 비교

직접 해보기 : 저장한 도시 복원

- 병원 확충을 선택하고 첫 달을 결산한 뒤 상태를 저장한다.
- 한 달 더 결산해 부채 155.88을 확인하고 앱을 종료한다.
- 저장한 JSON을 불러와 첫 달 부채 188로 돌아오는지 확인한다.
- 다시 한 달을 결산해 건설비 0 / 이자 1.88 / 부채 155.88을 확인한다.

정책 선택의 근거

- 목표 : 24시간 대기 감소 / 3개월 말 부채 110 이하
  - 네 정책의 결과에서 조건을 만족하는 대안을 찾는다.
- D 지역의 서비스까지 개선하려면 어떤 규칙과 비용이 더 필요할까?

오늘 구현한 도시

- 도로와 경제활동을 세금 / 비용 계산에 연결했다.
- 월 수지로 차입과 상환을 계산하고 현금 / 부채를 갱신했다.
- 같은 도시에서 정책을 바꾸고 서비스 성과와 재정을 비교했다.
- 월별 기록과 전체 상태를 저장해 결과를 설명하고 이어 실행했다.

감사합니다.

utilForever@gmail.com

https://github.com/utilForever

Facebook, Twitter, Instagram, LinkedIn: utilForever